

CodeRoute

Adaptive DSA Learning Platform

Complete Project Features, Specifications, Architecture, Database, API, Authentication & Deployment

Project	CodeRoute — full-stack adaptive DSA learning platform
Frontend	React + Vite
Backend	Spring Boot / Java REST API
Database	PostgreSQL
Security	Spring Security + JWT
Migrations	Flyway; current project state contains V1-V6
Deployment	Vercel frontend + Render backend

Purpose

CodeRoute brings DSA problem practice, topic organization, progress tracking, recommendations and supporting learning tools into one authenticated web application.

Documentation basis: current project information, observed API/configuration behavior, database checks, and the project requirements discussed during development. Details that were not directly verified are described at the feature level instead of being invented.

1. Project Overview

CodeRoute is a full-stack learning platform focused on Data Structures and Algorithms. The React frontend communicates with a Spring Boot backend. The backend owns authentication, business logic and database access, while PostgreSQL provides persistent storage and Flyway manages database changes.

Core architecture

Browser → Vercel React frontend → Render Spring Boot API → PostgreSQL

Primary goals

- Provide a structured DSA problem catalog instead of an unorganized list of questions.
- Organize problems by topics and difficulty.
- Track user attempts and topic progress.
- Use progress and recommendation APIs to support an adaptive learning experience.
- Provide secure account authentication using JWT.
- Keep the system deployable as separate frontend and backend services.

Typical user flow

- 1 User opens CodeRoute and signs up or signs in.
- 2 Backend validates the request and authenticates the account.
- 3 A JWT is returned after successful authentication.
- 4 Frontend stores the token and attaches it to protected API requests.
- 5 User accesses problems, topics, progress, recommendations and learning utilities.
- 6 Learning information is persisted in PostgreSQL.

2. Complete Feature List

Authentication & accounts

- Sign Up: create an account with name, email and password.
- Optional LeetCode Profile Link: registration accepts leetcodeProfileUrl; it is optional and is not added to Sign In.
- Sign In: authenticate an existing account.
- JWT authentication: successful authentication returns a JWT used for protected requests.
- Current user: the authenticated user's profile can be loaded through the auth API.
- Role: user records include a role; the observed application account uses USER.

DSA problem system

- Problem catalog: 170 problems are currently present in the verified database snapshot.
- Topics: 14 topics are currently present.
- Difficulty: problems use difficulty classifications; an observed example, Prefix Pivot Balance, is BEGINNER.
- Topic mapping: all 170 verified problems currently have a topic_id.
- Pagination: the frontend consumes paginated problem responses.
- Problem details: individual problem resources can be requested.
- Problem attempts: the schema contains problem_attempt tracking.

Learning & personalization

- User-topic progress: stored through user_topic_progress.
- Recommendations: frontend consumes /api/recommendations.
- Learning goals: the data model contains learning_goal.
- Adaptive direction: the platform combines learning state and problem/topic data to support personalized practice.

AI Mentor

- The project contains an AI Mentor/insights capability.
- The observed endpoint is /api/mentor/insights.
- This is separate from authentication and the core problem catalog.

DSA Todo

- The current project state contains a later migration, V6, for DSA Todo support.

UI & API experience

- React/Vite single-page frontend.
- Centralized API client for requests, Bearer-token attachment and error handling.
- Protected pages use the authenticated user context.
- Common HTTP failures are converted into user-readable API errors.

3. Technology Stack & Specifications

Layer	Technology	Responsibility
Frontend	React + Vite	UI, auth state, page behavior, API calls
API client	Fetch-based centralized client	HTTP, JSON, Bearer token, error mapping
Backend	Spring Boot / Java	REST endpoints, business logic, security
Security	Spring Security + JWT	Authentication and protected resources
Persistence	Spring Data JPA / Hibernate	Entity/database persistence
Database	PostgreSQL	Persistent application data
Migrations	Flyway	Versioned schema/data changes
Frontend hosting	Vercel	Production React hosting
Backend hosting	Render	Production Spring Boot hosting

Local development

Backend default port: 8080. Frontend uses the Vite development server. PostgreSQL is available locally through the installed PostgreSQL 18 client in the observed setup.

Production

The production frontend is configured to call the Render API through `VITE_API_BASE_URL`. The Render service uses environment variables for database, JWT and CORS configuration.

4. Database Specification

The current data model contains the following major tables/entities:

Table/entity	Purpose
app_user	User accounts, role data and optional LeetCode profile URL
learning_goal	User learning-goal information
topic	DSA topic catalog
problem	DSA problem catalog, difficulty and topic relationship
problem_attempt	User problem-attempt tracking
user_topic_progress	Progress by user and topic
DSA Todo data	Support introduced by the V6 migration

Verified data snapshot

Check	Observed value
Problems	170
Topics	14
Problems with topic_id	170
Example problem	Prefix Pivot Balance
Example difficulty	BEGINNER

LeetCode profile storage

The optional leetcodeProfileUrl value is part of the user data model. The V5 migration adds a nullable column to app_user. An observed authenticated-user response includes this field.

Flyway migration history

Version	Current documented purpose
V1-V4	Existing base/schema/problem migrations; preserved as existing migrations
V5	Adds nullable LeetCode profile URL to app_user
V6	Creates DSA Todo support

5. API Specification

Known API resources used/observed in the project

Endpoint	Purpose
/api/health	Backend health check
/api/auth/register	Registration and authentication
/api/auth/login	Login and authentication
/api/auth/me	Authenticated user information
/api/problems	Problem listing/pagination
/api/problems/{id}	Individual problem resource
/api/topics	Topic data
/api/users/me/progress	Current-user progress
/api/recommendations	Recommended learning/problem content
/api/mentor/insights	AI Mentor insights

Central API client behavior

- Uses VITE_API_BASE_URL in production and a localhost fallback for local development.
- Adds JSON Content-Type when a body is present.
- Adds Authorization: Bearer when a token is available.
- Parses JSON responses.
- Maps 401, 403, 404 and 503 responses to readable messages.
- Throws a specific network ApiError when the browser cannot reach the API.

Authentication request pattern

```
Authorization: Bearer <stored-jwt-token>
```

6. Authentication & Security Flow

- 1 User submits Sign Up or Sign In form.
- 2 AuthProvider calls the centralized API client.
- 3 API client sends JSON to `/api/auth/register` or `/api/auth/login`.
- 4 Backend validates credentials and account data.
- 5 On successful authentication, the backend returns a JWT and user information.
- 6 Frontend stores the JWT under `coderroute-token`.
- 7 Protected API calls automatically include the Bearer token.
- 8 Spring Security authenticates protected requests.

Important separation

The optional LeetCode profile URL is a registration/profile enhancement. It does not change the JWT design or the login request flow.

Production secrets

- `DB_URL`, `DB_USERNAME` and `DB_PASSWORD` configure PostgreSQL.
- `JWT_SECRET` configures JWT signing.
- JWT expiration is configured through the JWT expiration properties/environment variables.
- `FRONTEND_ORIGIN_PATTERNS` controls permitted browser origins for CORS.
- Production secrets must not be committed to the repository.

CORS

The backend uses an allowed-origin pattern configuration. The production Vercel origin must be included in `FRONTEND_ORIGIN_PATTERNS`. A browser CORS/preflight failure can prevent a perfectly healthy backend from being callable by the deployed frontend.

7. Frontend & Backend Responsibilities

Frontend responsibilities

- Render authentication and learning screens.
- Collect registration/login input.
- Maintain user/token state.
- Call backend APIs.
- Display problems, topics, progress and recommendations.
- Handle API/network errors.

Backend responsibilities

- Expose REST endpoints under /api.
- Authenticate users and protect endpoints.
- Create and validate JWT sessions.
- Persist users and learning data.
- Serve problem/topic/progress/recommendation data.
- Run Flyway migrations.
- Apply CORS rules.
- Expose health monitoring.

Configuration reference

Variable	Purpose
VITE_API_BASE_URL	Frontend API base URL
SERVER_PORT	Backend port; local default 8080
DB_URL	PostgreSQL JDBC URL
DB_USERNAME / DB_PASSWORD	PostgreSQL credentials
FLYWAY_ENABLED	Flyway enable/disable
FRONTEND_ORIGIN_PATTERNS	CORS allowed origins
JWT_SECRET	JWT signing secret
JWT_EXPIRATION / JWT_EXPIRATION_MS	JWT lifetime configuration
ANALYTICS_SERVICE_URL / TOKEN	Analytics service configuration

8. Deployment Specification

Production topology

Vercel React app → HTTPS → Render Spring Boot API → PostgreSQL

Vercel

- Hosts the production frontend.
- VITE_API_BASE_URL should point to the Render API origin.
- Changing a VITE_* variable requires a new Vercel deployment/build.

Render

- Hosts the Spring Boot backend service.
- Observed service: coderoute-api.
- Observed health endpoint: <https://coderoute-api.onrender.com/api/health>.
- Uses environment variables for DB, JWT and CORS settings.

Production verification

- 1 Check the Render health endpoint and confirm HTTP 200 with status UP.
- 2 Open the Vercel application.
- 3 Create a test account.
- 4 Sign in.
- 5 Open Problems and verify data loads.
- 6 Check progress/recommendations.
- 7 If the browser shows CORS preflight failure, check FRONTEND_ORIGIN_PATTERNS.
- 8 If the frontend says it cannot reach the API, check VITE_API_BASE_URL and redeploy.

Local verification commands

```
cd "C:\Users\hp\OneDrive\Desktop\CodeRoute - End Term Project Mern\backend"\n.\mvnw.cmd spring-boot:run
```

```
Invoke-WebRequest http://localhost:8080/api/health -UseBasicParsing
```

```
cd "C:\Users\hp\OneDrive\Desktop\CodeRoute - End Term Project Mern\frontend"\nnpm install\nnpm run build\nnpm run dev
```

9. Testing & Verification Checklist

Area	Expected result
Backend build	Maven build/test completes successfully
Backend health	GET /api/health returns HTTP 200 and status UP
Database	PostgreSQL connection works and problem data is present
Registration	New account can be created; optional LeetCode URL can be saved
Login	Existing account can sign in and receive authentication data
JWT	Protected requests contain Authorization Bearer token
Problems	Problem list loads from API; verified catalog has 170 problems
Topics	Topic data loads; verified database has 14 topics
Progress	Authenticated progress endpoint responds
Recommendations	Recommendation API responds when its service/data is available
Frontend build	npm run build completes successfully
Deployment	Vercel can call Render without CORS/network errors

Useful PostgreSQL check

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -d coderoute -c "SELECT COUNT(*) FROM problem;"
```

Verified current result: 170 problems.

10. Viva / Handoff Explanation

What is CodeRoute?

CodeRoute is a full-stack adaptive DSA learning platform. It lets users create accounts, authenticate securely, practice a structured set of DSA problems, learn by topic and difficulty, track progress, and receive recommendations.

Why React + Spring Boot + PostgreSQL?

React provides a responsive client UI; Spring Boot provides a structured REST API and security layer; PostgreSQL provides reliable relational persistence; Flyway makes database changes version-controlled.

How does login work?

The frontend sends credentials to the backend. After successful authentication, the backend returns a JWT. The frontend stores the token and sends it in the Authorization Bearer header on protected API calls.

How does the DSA system work?

Problems are stored in PostgreSQL with difficulty and topic relationships. User attempts and topic progress are stored separately, allowing the application to represent learning state and power recommendation/progress features.

What is the LeetCode field?

It is an optional profile URL collected during Sign Up and persisted as leetcodeProfileUrl. It is deliberately kept separate from the login/JWT flow.

How is it deployed?

The React frontend is deployed on Vercel, while the Spring Boot API is deployed on Render. The backend connects to PostgreSQL and permits the production frontend through CORS.

Live demo sequence

- 1 Open the deployed site.
- 2 Register a new user.
- 3 Optionally enter a LeetCode profile URL.
- 4 Sign in.
- 5 Show the authenticated application.
- 6 Open Problems and demonstrate filtering/pagination/topic information.
- 7 Show progress/recommendations.
- 8 Explain the JWT in a protected request.
- 9 Show the backend health endpoint.

11. Quick Reference

Item	Value
Project	CodeRoute — Adaptive DSA Learning Platform
Frontend	React + Vite
Backend	Spring Boot / Java
Database	PostgreSQL
Migrations	Flyway V1-V6 in current project state
Authentication	Spring Security + JWT
Local API	http://localhost:8080
Production API	https://coderoute-api.onrender.com
Production frontend	https://mern-end-term-project-term-5-5umb.vercel.app
Problem count	170 verified in current database snapshot
Topic count	14 verified in current database snapshot
Token key	coderoute-token
Frontend API variable	VITE_API_BASE_URL
CORS variable	FRONTEND_ORIGIN_PATTERNS
DB variables	DB_URL, DB_USERNAME, DB_PASSWORD
JWT variables	JWT_SECRET and expiration setting

Final project summary

CodeRoute is a complete full-stack DSA learning application with secure authentication, a PostgreSQL-backed problem catalog, topics and difficulty, attempt/progress tracking, recommendations, AI Mentor support, DSA Todo support, optional LeetCode profile data, and a Vercel-to-Render production deployment model.

A reader who understands the sections above should be able to explain the project's purpose, users, major features, data model, request flow, authentication, deployment, configuration and verification process without needing the original development conversation.